

Применение нейронных сетей в задачах обработки естественного языка

ЛЕКЦИЯ №8

Темы

1. Обработка текста
2. Плотные векторные представления
3. Примеры

Применение нейронных сетей в задачах обработки естественного языка

1. Анализ текста.

2. Синтез речи.

Задача типа TextToSpeech. На вход нейронной сети подается произвольный текст, на выходе модель должна синтезировать голосовой фрагмент, соответствующий исходному текстовому образцу.

3. Распознавание речи.

Задача типа SpeechToText (обратная предыдущей). На вход нейронной сети приходит голосовой фрагмент, на выходе модель должна давать текстовое представление голосового образца. На сегодняшний день существует довольно много разновидностей моделей данного типа. Есть как платные варианты (от компаний Google, Yandex, Facebook), так и бесплатные (например, архитектура DeepSpeech).

4. Генерация текста.

«Выявить закономерности в имеющемся тексте и создать новый «осмысленный» текст».

Область применения

- Диалоговые системы
- Машинные переводы
- CMS, Web-оптимизация (уникальные тексты для сайтов)
- Умные помощники
- Генерация литературных произведений, стихов, шуток
- И т.д...

Общая постановка задачи

- Собрать подходящую базу (тексты, диалоги, шаблоны и т.п)
- Предобработка (очистка от мусора, токенизация)
- Преобразовать в машинный язык (числа, последовательности, вектора)
- Выбранным алгоритмом получить результат
- Результат преобразовать обратно в естественный язык

Обработка текстов

- **Токенизация. Tokenize the Sentences**
- **Создание словаря частотности. Create a Dictionary of Word Frequency**
- **Векторизация текста:**
 - простые модели Bag of Words Model, ONE ,
 - плотные векторные представления Word Embedding (Word2Vec, GloVe, FastText)
- **Морфологический анализ Py morphology2**

Gensim — библиотека обработки естественного языка, предназначенная для «Тематического моделирования».

Обработка текстов

1. Токенизация

а б в г д ж з ... я

0 1 2 3 4 5 6 ... 32

д я д я

4 32 4 32

Tokenizer:

word counts : словарь.

word docs : целое число – количество документов, которые используются для токенайзера.

word index : A dictionary of words and their uniquely assigned integers.

document count : A dictionary of words and how many documents each appeared in.

Обработка текстов

2. Создание словаря частотности.

Библиотека Keras содержит метод `tokenizer()` - принимает на вход текст, а возвращает словарь частотности.

Фильтрует "мусорные" символы, составит множество пар "слово - количество повторений слова", расположит все полученные значения обратном порядке.

Самые частые слова текста (союзы, частицы, предлоги, местоимения) займут место в начале списка индексов, а самые редкие окажутся в конце.

и	2	марк	39585
в	3	корнуэлл	132234
не	4	ел	4028
я	5	хлеб	2866
что	6	с	8
на	7	сыром	19591
с	8	когда	40
он	9	раздался	1295
а	10	стук	2723
как	11	в	3
то	12	дверь	181
это	13	комната	2001
но	14	была	95
все	15	маленькой	2174
у	16	и	2
по	17	холодной	2950
...	...	горстка	108963
раздался	1295	горевших	120123
...	...	прутьев	59542
комната	2001	в	3
...	...	маленьком	6276
зияющая	133072	камине	10454

Обработка текстов. Токенизация

Пример создания токенайзера

```
tokenizer = Tokenizer(num_words=5, filters='!"#$%&()*+,-—./...:;<=>?@[\\]^_`{|}~«»\t\n\xa0\ufeff', lower=True, split=' ',  
oov_token='неизвестное_слово', char_level=False)
```

параметры

num_words - объем словаря

filters - убираемые из текста ненужные символы

lower - приведение слов к нижнему регистру

split - разделитель слов

char_level - указание разделять по словам, а не по единичным символам

oov_token - токен для слов, которые не вошли в словарь

Построение частотного словаря по текстам

```
tokenizer.fit_on_texts(sample_text)
```

Обработка текстов. Токенизация

Пример создания токенайзера

```
tokenizer = Tokenizer(num_words=5, filters='!"#$%&()*+,-—./...:;<=>?@[\\]^_`{|}~«»\t\n\xa0\ufeff', lower=True, split=' ',  
oov_token='неизвестное_слово', char_level=False)
```

параметры

num_words - объем словаря

filters - убираемые из текста ненужные символы

lower - приведение слов к нижнему регистру

split - разделитель слов

char_level - указание разделять по словам, а не по единичным символам

oov_token - токен для слов, которые не вошли в словарь

Построение частотного словаря по текстам

```
tokenizer.fit_on_texts(sample_text)
```

Обработка текстов. Токенизация

Построение частотного словаря по текстам `tokenizer.fit_on_texts(sample_text)`

Метод `word_index` для словаря частотности выводит все пары по типу: «Слово: индекс частоты повторений слова».

```
from keras.preprocessing.text import Tokenizer # define 5 documents
docs = ["Было темно", "Жечь бумагу было наслаждением.", " Какое-то особое наслаждение видеть"]
tokenizer = Tokenizer()
# fit the tokenizer on the documents
tokenizer.fit_on_texts(docs)
```

```
tokenizer.word_counts, tokenizer.document_count, tokenizer.word_index, tokenizer.word_docs
```

```
(OrderedDict([('было', 2),
              ('темно', 1),
              ('жечь', 1),
              ('бумагу', 1),
              ('наслаждением', 1),
              ('какое', 1),
              ('то', 1),
              ('особое', 1),
              ('наслаждение', 1),
              ('видеть', 1)]),
```

```
3,
{'было': 1,
 'темно': 2,
 'жечь': 3,
 'бумагу': 4,
 'наслаждением': 5,
 'какое': 6,
 'то': 7,
 'особое': 8,
 'наслаждение': 9,
 'видеть': 10},
defaultdict(int,
              {'было': 2,
               'темно': 1,
               'бумагу': 1,
               'жечь': 1,
               'наслаждением': 1,
               'то': 1,
               'видеть': 1,
               'особое': 1,
               'какое': 1,
               'наслаждение': 1}))
```

Обработка текстов.

Преобразование исходного словаря с текстом в последовательность индексов слов - функция `texts_to_sequences()`:

```
# Словарь токенизатора с индексами слов
tokenizer.word_index
```

```
{'восемь': 9,
 'два': 5,
 'неизвестное_слово': 1,
 'один': 6,
 'пять': 2,
 'семь': 8,
 'три': 4,
 'четыре': 3,
 'шесть': 7}
```

```
# Представление словаря в виде списка пар (слово, индекс)
list(tokenizer.word_index.items())
```

```
[('неизвестное_слово', 1),
 ('пять', 2),
 ('четыре', 3),
 ('три', 4),
 ('два', 5),
 ('один', 6),
 ('шесть', 7),
 ('семь', 8),
 ('восемь', 9)]
```

Векторизация текста

Классический подход классификации текста **Bag of Words Model**

1. Составление словаря

$\mathbf{x}^{[1]}$ = "The sun is shining"

$\mathbf{x}^{[2]}$ = "The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining,
the weather is sweet, and one and one is two"

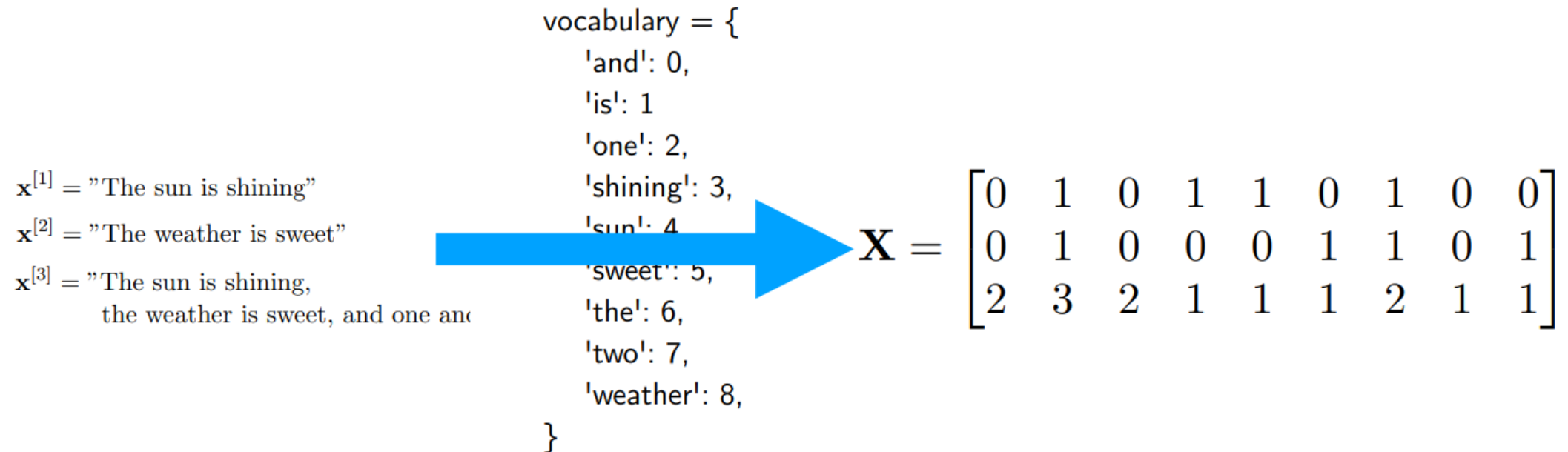


```
vocabulary = {  
    'and': 0,  
    'is': 1  
    'one': 2,  
    'shining': 3,  
    'sun': 4,  
    'sweet': 5,  
    'the': 6,  
    'two': 7,  
    'weather': 8,  
}
```

Векторизация текста

Классический подход классификации текста **Bag of Words Model**

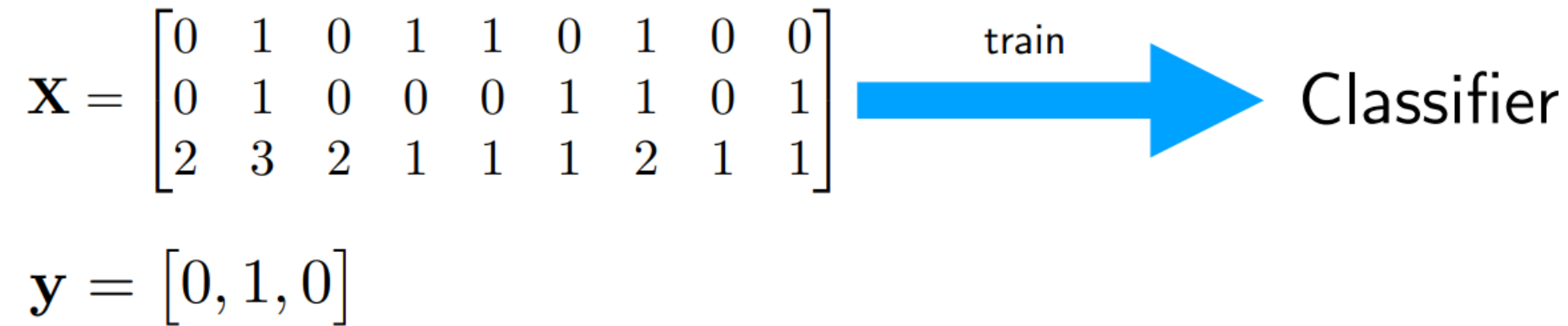
2. На основе словаря дата сет преобразуется в Bag of Words матрицу
(размерность матрицы = размерность словаря)



Векторизация текста

Классический подход классификации текста **Bag of Words Model**

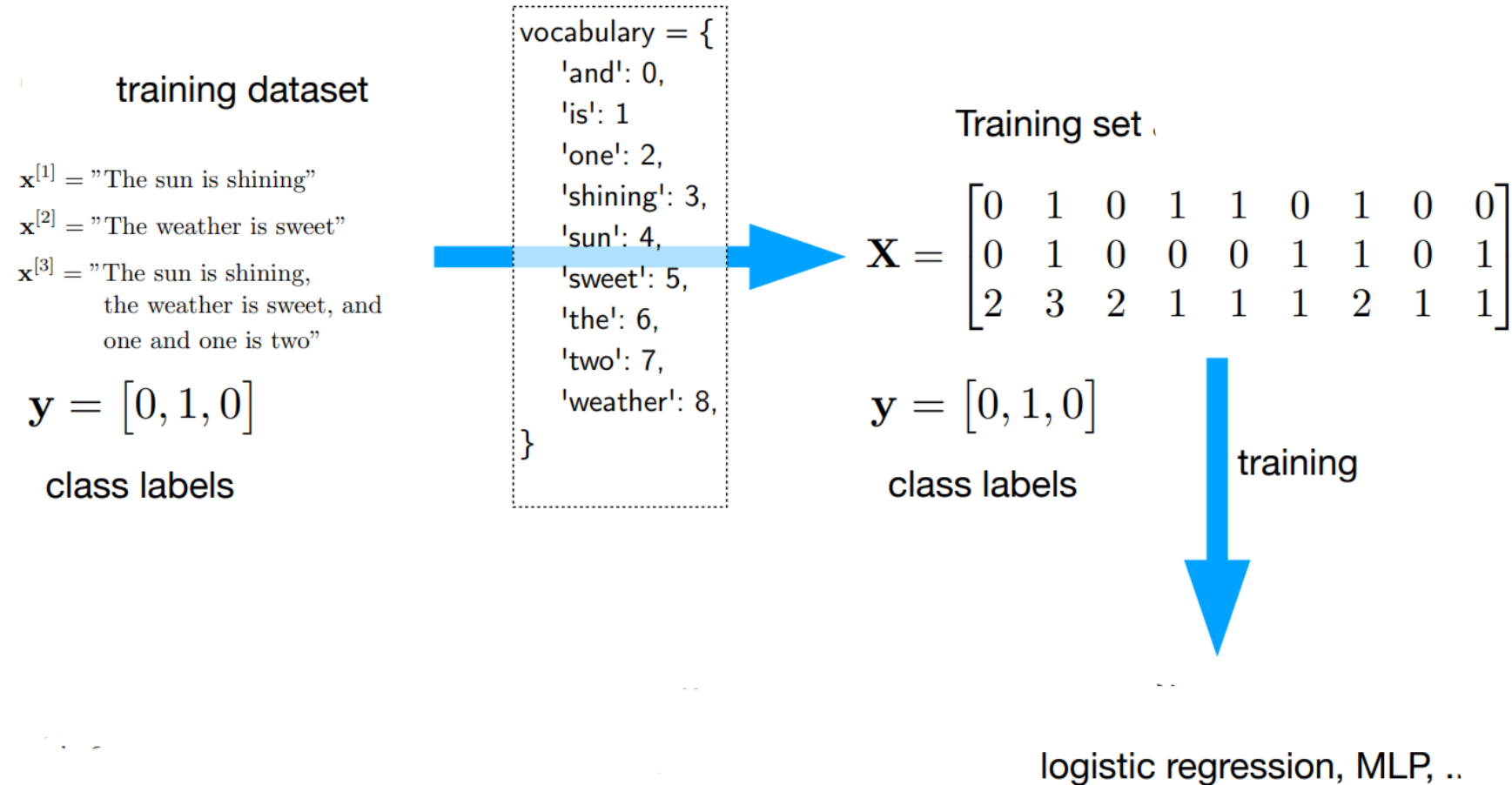
3. Обучение модели на основе Bag of Words



Векторизация текста

Классический подход классификации текста **Bag of Words Model**

3. Обучение модели на основе Bag of Words



Векторизация текста

Классический подход классификации текста **Bag of Words Model**

3. Обучение модели на основе Bag of Words

$\mathbf{x}^{[1]}$ = "The sun is shining"

$\mathbf{x}^{[2]}$ = "The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining,
the weather is sweet, and one and one is two"

Векторизация текста

Классический подход классификации текста **Bag of Words Model**

4 . Обучение модели на основе Bag of Words

Создание n-gramm ($n > 1$)

1 token = 1 word:

$\mathbf{x}^{[1]} = \text{"The sun is shining"}$

1 token = 2 words:

$\mathbf{x}^{[1]} = \text{"The sun is shining"}$

Обработка текстов. Векторизация текста

Метод преобразования списка из индексов слов в матрицу Bag Of Words

Преобразование списка индексов в Bag Of Words происходит методом `sequences_to_matrix()`.

Если "texts_to_sequences" переводится как «тексты в последовательности»,

то "sequences_to_matrix" – как «последовательности в матрицу».

Получим тренировочную выборку в формате Bag Of Words.

Параметры:

`binary`: присутствует слово в документе (по умолчанию).

`count`: количество слов в документе.

`tfidf`: Text Frequency-Inverse DocumentFrequency (TF-IDF) считает для каждого слова в документе.

`freq`: частотность слов.

```
# Представление списка подпоследовательностей в виде разреженной матрицы:  
# каждая подпоследовательность представляется как вектор bag of words  
tokenizer.sequences_to_matrix(x_train)
```

```
array([[0., 1., 0., 0., 1.],  
       [0., 1., 1., 1., 0.],  
       [0., 0., 0., 1., 1.],  
       [0., 0., 1., 1., 0.],  
       [0., 0., 1., 0., 0.],  
       [0., 1., 0., 0., 0.]])
```

Обработка текстов. Токенизация

Параметры

binary: присутствует слово в документе (по умолчанию).

count: количество слов в документе.

tfidf: Text Frequency-Inverse DocumentFrequency (TF-IDF) считает для каждого слова в документе.

freq: частотность слов.

```
encoded_docs = tokenizer.texts_to_matrix(docs, mode='freq')
encoded_docs
```

```
array([[0.   , 0.5  , 0.5  , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   ],
       [0.   , 0.25, 0.   , 0.25, 0.25, 0.25, 0.   , 0.   , 0.   , 0.   , 0.   ],
       [0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.2  , 0.2  , 0.2  , 0.2  , 0.2  ]])
```

```
encoded_docs = tokenizer.texts_to_matrix(docs, mode='tfidf')
encoded_docs
```

```
array([[0.          , 0.69314718, 0.91629073, 0.          , 0.          ,
        0.          , 0.          , 0.          , 0.          , 0.          ,
        0.          ],
       [0.          , 0.69314718, 0.          , 0.91629073, 0.91629073,
        0.91629073, 0.          , 0.          , 0.          , 0.          ,
        0.          ],
       [0.          , 0.          , 0.          , 0.          , 0.          ,
        0.          , 0.91629073, 0.91629073, 0.91629073, 0.91629073,
        0.91629073]])
```

Обработка текстов

```
sample_text = ['один два три четыре пять два Три три четыре четыре четыре пять пять пять пять шесть семь восемь']
tokenizer = Tokenizer(num_words=5, filters='!"#$%&()*+,-./...:;<=>?@[\\]^_`{|}~«»\t\n\ха0\ueff', lower=True, split=' ', oov_token='неизвестное_слово', char_level=False)

# Использованы параметры:
# num_words - объем словаря
# filters - убираемые из текста ненужные символы
# lower - приведение слов к нижнему регистру
# split - разделитель слов
# char_level - указание разделять по словам, а не по единичным символам
# oov_token - токен для слов, которые не вошли в словарь

# Построение частотного словаря по текстам
tokenizer.fit_on_texts(sample_text)
```

```
encoded_docs = tokenizer.texts_to_matrix(docs, mode='count')
encoded_docs
```

```
array([[0., 2., 0., 0., 0.],
       [0., 4., 0., 0., 0.],
       [0., 5., 0., 0., 0.]])
```

```
encoded_docs = tokenizer.texts_to_matrix(docs, mode='tfidf')
encoded_docs
```

```
array([[0.          , 1.17360019, 0.          , 0.          , 0.          ],
       [0.          , 1.65405321, 0.          , 0.          , 0.          ],
       [0.          , 1.80872453, 0.          , 0.          , 0.          ]])
```

Создание выборки

Обучение модели на основе Bag of Words.

Создание n-gramm ($n > 1$)

Очнувшись, я несколько времени не мог опомниться и не понимал, что со мною случилось. Я лежал на кровати, в незнакомой горнице, и чувствовал большую слабость.....

175 3 45 23 5 45 454 2 5 456 21 7 78 74 3 78 10 45 4 89 786 2 64 71
123

ширина_окна = 5

шаг = 2

175	3	45	23	5
45	23	5	45	454
5	45	454	2	5
454	2	5	456	21

Плотные векторные представления **Word Embadding**

Дистрибутивная гипотеза

- Лингвистические единицы из схожих контекстов имеют близкое расстояние

Лингвистические единицы

- Слова
- Словосочетания

Контекст

- Скользящее окно
- Документ

Модели

- Word2Vec
- GloVe
- FastText

Обучение

- Обучение с учителем
- Слой с плотным векторным представлением – часть сети
- Инициализация векторов случайными значениями
- Обучение с помощью метода обратного распространения ошибки

Плотные векторные представления **Word Embedding**

Embedding - "умная нормализация".

Каждый токен преобразуем в вектор вещественных чисел при помощи обучения.

Embedding-слой создаёт скрытое пространство, внутри которого для каждого индекса слова будет создан свой отдельный вектор с новыми значениями.

С каждым словом w_i словаря V связан

вектор $u_i = \{ \}$ так, чтобы семантически близкие слова имели схожие векторы. В результате этого, например, слова dog, cat, rabbit в векторном пространстве должны оказаться рядом (образовать кластер) и при этом быть в отдалении от кластера слов car, bus, train.

Метрика **pPMI** (positive pointwise mutual information):

$$\text{pPMI} = \max(0, \text{PMI}), \quad \text{PMI} = \log \frac{P(w_1, w_2)}{P(w_1)P(w_2)} \equiv \log \frac{P(w_1 | w_2)}{P(w_1)}.$$

	paw	barks	eats	sleeps	drives	gas	wheel	red	white	did	the
cat	red	white	white	red	white	white	white	white	orange	red	dark red
dog	red	red	red	orange	white	white	white	white	white	red	dark red
car	white	white	white	white	red	red	red	red	red	red	dark red
bus	white	white	white	white	red	red	red	orange	white	red	dark red

Плотные векторные представления Word Embedding

Embedding - "умная нормализация".

1. Мера близости – семантическое сходство слов

евклидово расстояние: $\text{dist}_1(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{u} - \mathbf{v})^2}$

косинусное расстояние: $\text{dist}_2(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}| \cdot |\mathbf{v}|}$

dist_cos									dist_len							
	cat	car	apple	gin	monday	red	small	say	cat	car	apple	gin	monday	red	small	say
cat		0.94	1.01	0.90	1.07	1.02	0.84	1.01		2.62	2.62	1.98	2.27	3.06	2.46	2.32
car	0.94		0.97	1.07	1.06	0.87	1.02	1.02	2.62		2.57	2.10	2.26	2.83	2.73	2.33
apple	1.01	0.97		0.99	0.94	0.84	0.94	1.03	2.62	2.57		1.90	2.02	2.70	2.51	2.22
gin	0.90	1.07	0.99		1.16	0.90	1.01	1.09	1.98	2.10	1.90		1.40	2.38	2.03	1.53
monday	1.07	1.06	0.94	1.16		0.89	1.09	1.15	2.27	2.26	2.02	1.40		2.48	2.27	1.81
red	1.02	0.87	0.84	0.90	0.89		0.98	1.05	3.06	2.83	2.70	2.38	2.48		2.98	2.73
small	0.84	1.02	0.94	1.01	1.09	0.98		1.04	2.46	2.73	2.51	2.03	2.27	2.98		2.33
say	1.01	1.02	1.03	1.09	1.15	1.05	1.04		2.32	2.33	2.22	1.53	1.81	2.73	2.33	

Плотные векторные представления Word Embedding

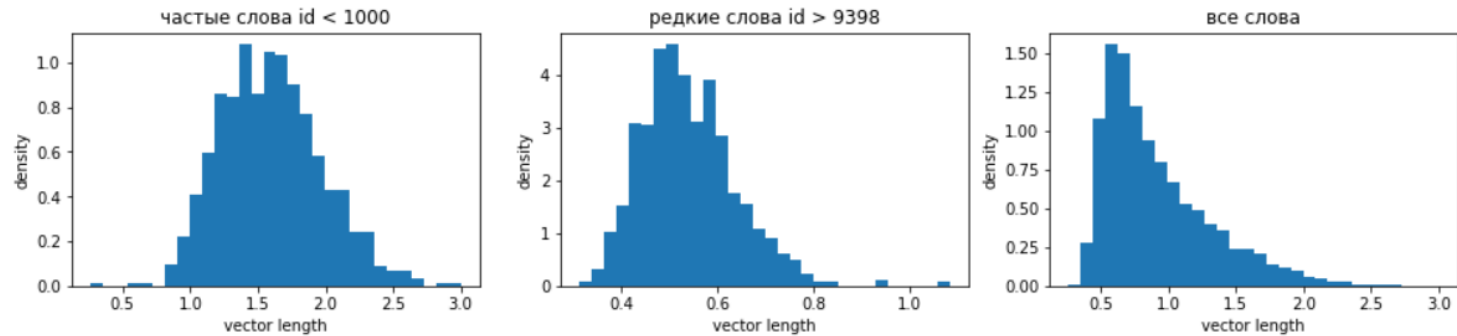
Embedding - "умная нормализация".

1. Мера близости – семантическое сходство слов

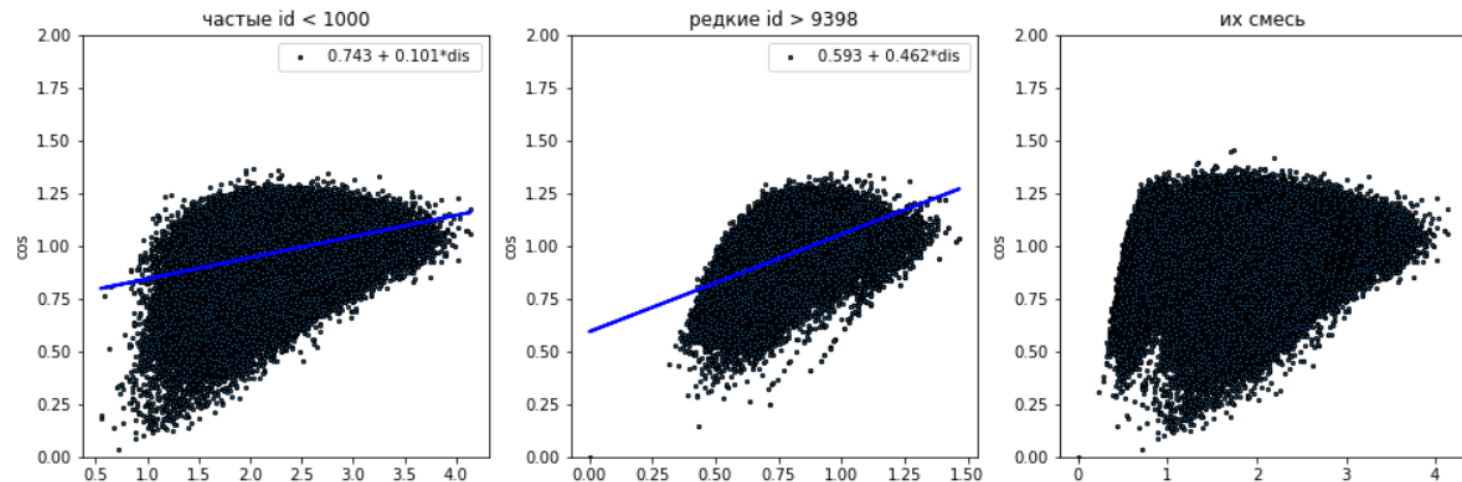
евклидово расстояние: $\text{dist}_1(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{u} - \mathbf{v})^2}$

косинусное расстояние: $\text{dist}_2(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}| \cdot |\mathbf{v}|}$

гистограммы распределения длин векторов по отдельности для частых слов (первые 1000), редких слов и всех вместе. Частые слова имеют более длинные векторы, что типично для всех методов эмбединга:



зависимость евклидового расстояния (корень из суммы квадратов) от косинусного $1 - \cos$ для первых 1000 и последних 1000 (по частотности) слов:



Плотные векторные представления **Word Embedding**

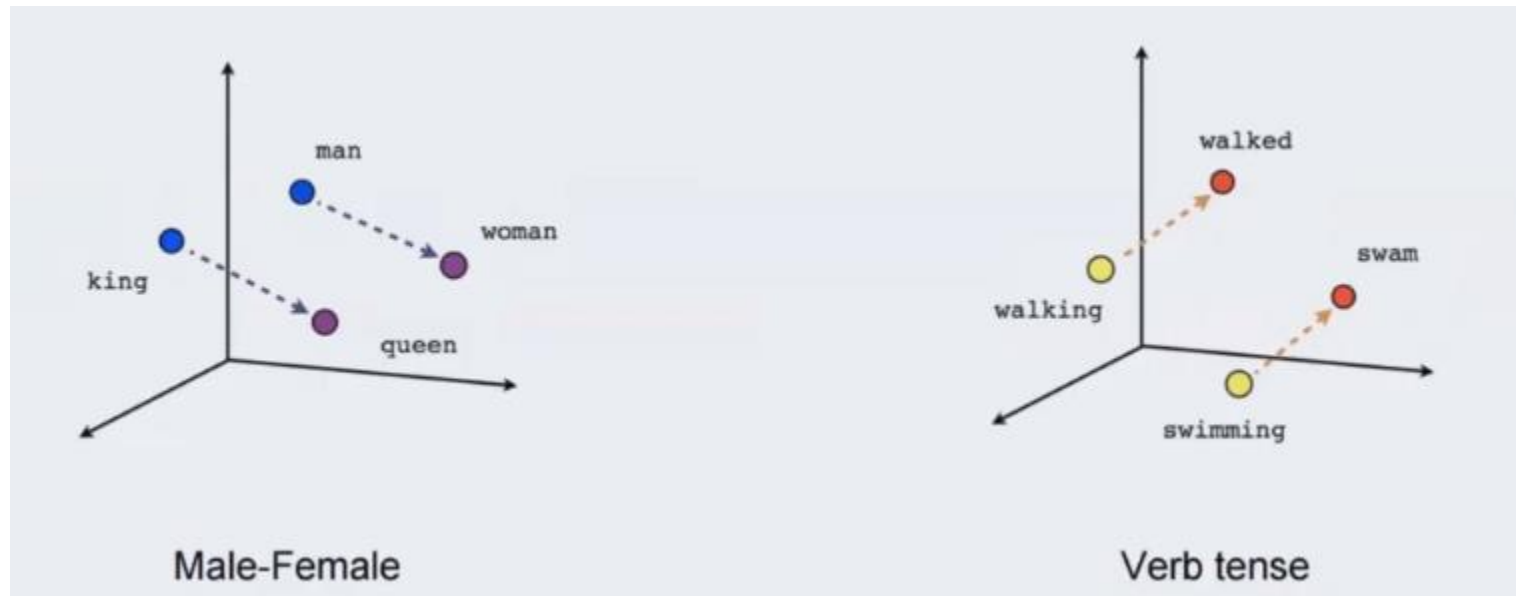
Существуют **предобученные Embedding** на английском:

Glove (1000 - мерный)

Word2Vec (400 - мерный)

FastText

На русском - Elmo, Bert.



GloVe: глобальные векторные представления

GloVe: Global Vectors for Word Representation

GloVe — это алгоритм обучения без учителя для получения векторных представлений слов.

Обучение выполняется на агрегированной глобальной статистике совпадения слов из корпуса, и полученные представления демонстрируют интересные линейные подструктуры векторного пространства слов.

1. Мера близости – семантическое сходство слов

Евклидово расстояние (или косинусное сходство) между двумя векторами слов обеспечивает эффективный метод измерения лингвистического или семантического сходства соответствующих слов

Например, слова, наиболее близкие к целевому слову лягушка :

лягушка
лягушки
жаба
литория
лептодактилиды
рана
ящерица



3. литория



4. лептодактилиды



5. рана



7. элеутеродактиль

GloVe: глобальные векторные представления

GloVe: Global Vectors for Word Representation

2. Линейные подконструкции

Метрики подобия, используемые для оценки ближайших соседей, дают один скаляр, который количественно определяет родство двух слов.

- Для английского языка можно загрузить готовые векторизации слов с различными размерностями $E_DIM = E$, например Glove. Файл Glove это обычный текстовый файл, загрузка которого имеет вид:

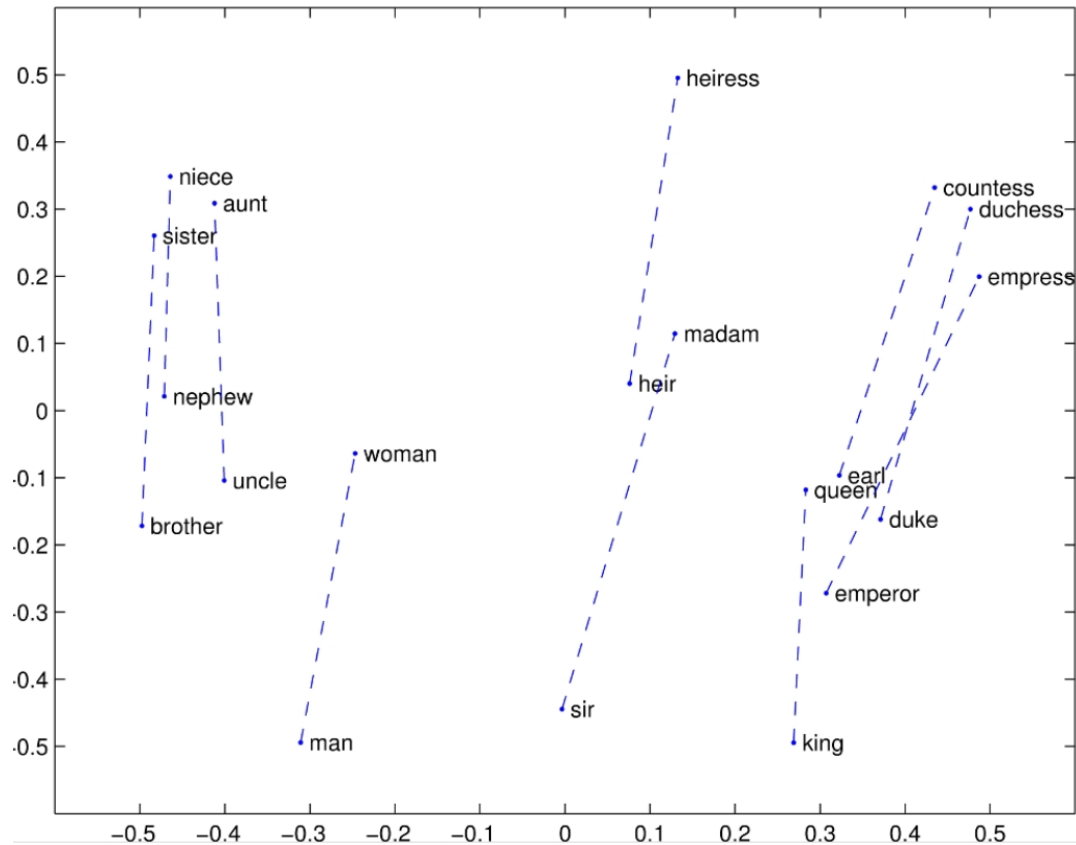
```
vec = {}  
with open('glove.6B.100d.txt', "r", encoding='utf-8') as file:  
    for line in file:  
        values = line.split()  
        vec[ values[0] ] = np.asarray(values[1:], dtype='float32')
```

Этот файл содержит 400000 слов, отсортированных по частоте. Первые 100 приведены ниже (разделены чертой):

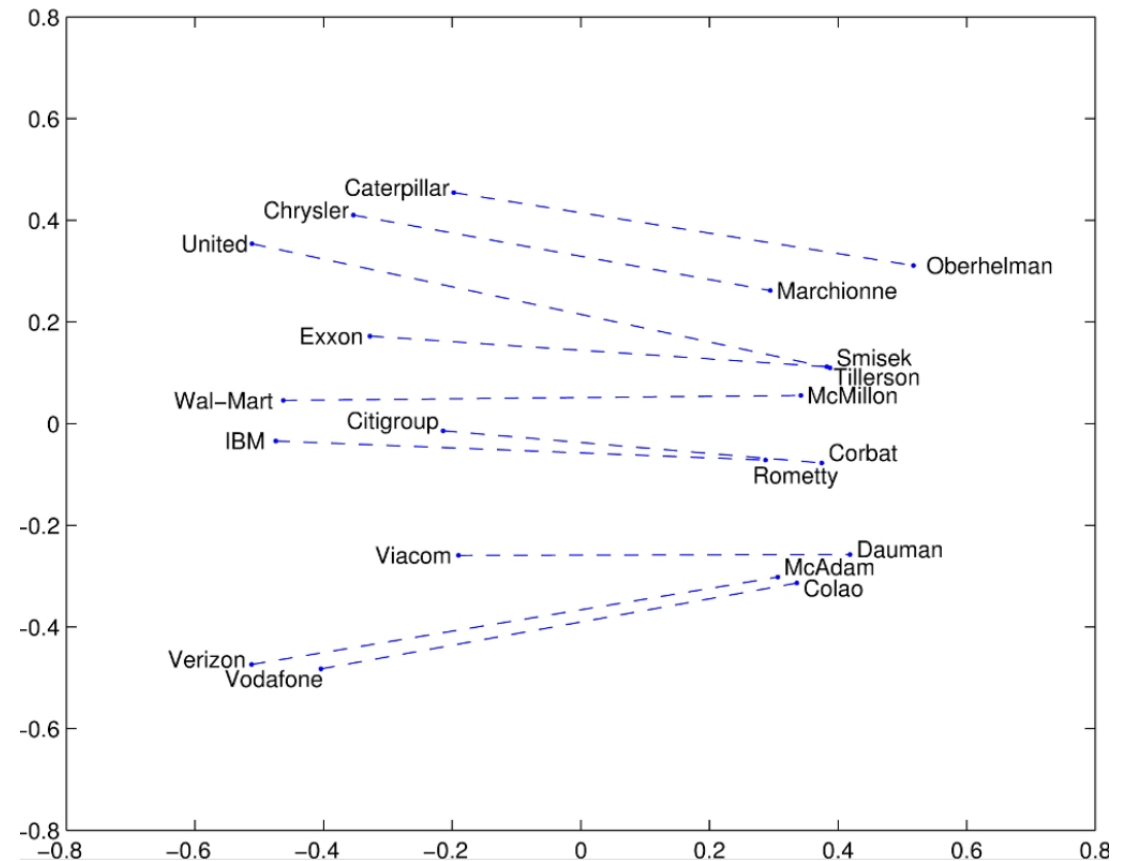
```
|the|.|.|of|to|and|in|a|'|s|for|_|that|on|is|was|said|with|he|as|it|by|at|(|)|from|his|'|  
|`|an|be|has|are|have|but|were|not|this|who|they|had|i|which|will|their|_|or|its|one|after|  
|new|been|also|we|would|two|more|'|first|about|up|when|year|there|all|--|out|she|other|  
|people|n't|her|percent|than|over|into|last|some|government|time|$|you|years|if|no|world|  
|can|three|do|_|president|only|state|million|could|us|most|_|against|u.s.|
```

GloVe: глобальные векторные представления

GloVe: Global Vectors for Word Representation



Мужчина и женщина



Компания – генеральный директор

GloVe: Global Vectors for Word Representation Jeffrey Pennington, Richard Socher, Christopher D. Manning
Computer Science Department, Stanford University, Stanford, CA 94305

GloVe: глобальные векторные представления

GloVe: Global Vectors for Word Representation

Обучение

Модель GloVe обучается на ненулевых элементах глобальной матрицы совпадения слов, которая показывает, как часто слова встречаются друг с другом в данном корпусе.

В этом подходе сначала вычисляются логарифмы вероятностей P_{ij} совместной встречаемости слов w_i и w_j в тексте внутри скользящего окна из 10 слов. Подбираемыми параметрами модели являются компоненты E -мерных векторов $\mathbf{u}_i$ слов и параметры смещения b_i . В процессе обучения минимизируется следующий функционал ошибки:

$$\text{Loss} = \sum_{ij} f(N_{ij}) (\mathbf{u}_i \mathbf{u}_j' + b_i + b_j' - \log P_{ij})^2 = \min_{\mathbf{u}_i, \mathbf{u}_j', b_i, b_j'}$$

GloVe: глобальные векторные представления

GloVe: Global Vectors for Word Representation

Пример

Рассмотрим вероятности совпадения целевых слов «лед» и «пар» с различными контрольными словами из словаря. Вот некоторые фактические вероятности из 6 миллиардов слов:

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

Лед чаще встречается с твердым телом , чем с газом , тогда как пар чаще встречается с газом , чем с твердым телом . Оба слова часто встречаются вместе с их общим свойством «вода» , и оба нечасто встречаются вместе с несвязанным словом «мода» .

Пример. Классификация текстов

Класс: Стругацкие

train: Парень из преисподней 1 Ну и деревня! Сроду я таких деревень не видел и не знал даже, что такие деревни бывают. Дома круглые,
test : ОТЕЛЬ «У ПОГИБШЕГО АЛЬПИНИСТА» ГЛАВА 1 Я остановил машину, вылез и снял черные очки. Все было так, как рассказывал Згут. Отель

Класс: Макс Фрай

train: Власть несбывшегося – С тех пор как меня угораздило побывать в этой грешной Черхавле, мне ежедневно снится какая-то дичь! – сердито
test : Слишком много кошмаров Когда балансируешь над пропастью на узкой, скользкой от крови доске, ответ на закономерный вопрос: «Как мен»

Класс: Булгаков

train: Белая гвардия Посвящается[1] Любви Евгеньевне Белозерской[2] Пошел мелкий снег и вдруг повалил хлопьями. Ветер завыл; сделалас
test : Дон Кихот ДЕЙСТВУЮЩИЕ ЛИЦА Алонсо Кихано, он же Дон Кихот Ламанчский. Антония – его племянница. Ключница Дон Кихота. Санчо Панса –

Класс: Клиффорд Саймак

train: Всё живое... Когда я выехал из нашего городишка и повернул на шоссе, позади оказался грузовик. Этакая тяжелая громадина с прицепом
test : Зачарованное паломничество 1 Гоблин со стропил следил за прячущимся монахом, который шпионил за ученым. Гоблин ненавидел монаха и

Класс: Рэй Брэдберри

train: 451° по Фаренгейту ДОНУ КОНГДОНУ С БЛАГОДАРНОСТЬЮ Если тебе дадут линованную бумагу, пиши поперёк. Хуан Рамон Хименес Часть 1
test : Марсианские хроники МОЕЙ ЖЕНЕ МАРГАРЕТ С ИСКРЕННЕЙ ЛЮБОВЬЮ «Великое дело – способность удивляться, – сказал философ. – Космические

Класс: О. Генри

train: «Лиса-на-рассвете» Коралио нежился в полуденном зное, как томная красавица в сурово хранимом гареме. Город лежал у самого моря на п
test : Багдадская птица Без всякого сомнения, дух и гений калифа Гаруна аль-Рашида осенил маркграфа Августа-Михаила фон Паульсена Квигга.

Пример. Классификация текстов

```
[ ] # Задание параметров преобразования
VOCAB_SIZE = 20000          # Объем словаря для токенизатора
WIN_SIZE    = 1000          # Длина отрезка текста (окна) в словах
WIN_HOP     = 100           # Шаг окна разбиения текста на векторы
```

Токенизация и преобразование в последовательности

```
# Токенизация и построение частотного словаря по обучающим текстам
# Используется встроенный в Keras токенизатор для разбиения текста и построения частотного словаря
tokenizer = Tokenizer(num_words=VOCAB_SIZE, filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~«»\t\n\x00\ufe0f',
                      lower=True, split=' ', oov_token='неизвестное_слово', char_level=False)

# Использованы параметры:
# num_words    - объем словаря
# filters      - убираемые из текста ненужные символы
# lower        - приведение слов к нижнему регистру
# split        - разделитель слов
# char_level   - указание разделять по словам, а не по единичным символам
# oov_token    - токен для слов, которые не вошли в словарь

# Построение частотного словаря по обучающим текстам

tokenizer.fit_on_texts(text_train)

# Построение словаря в виде пар слово - индекс
items = list(tokenizer.word_index.items())
```

Пример. Классификация текстов

```
# Преобразование обучающих и проверочных текстов в последовательность индексов согласно частотному словарю
seq_train = tokenizer.texts_to_sequences(text_train)
seq_test = tokenizer.texts_to_sequences(text_test)

print("Фрагмент обучающего текста:")
print("В виде оригинального текста: ", text_train[1][:101])
print("Он же в виде последовательности индексов: ", seq_train[1][:20])
```

Фрагмент обучающего текста:

В виде оригинального текста: Власть несбывшегося – С тех пор как меня угораздило побывать в этой грешной Черхавле, мне ежеднев

Он же в виде последовательности индексов: [1508, 1, 8, 282, 236, 11, 24, 10104, 6015, 3, 144, 7863, 1, 21, 3627, 12947, 238, 12, 18834, 1301]

Пример. Классификация текстов

```
# Функция вывода статистики по текстам
def print_text_stats(title, texts, sequences, class_labels=CLASS_LIST):
    # Суммарное количество символов и слов в тексте
    chars = 0
    words = 0

    print(f'Статистика по {title} текстам:')

    # Вывод итогов по всем классам данного набора текстов и их последовательностей индексов
    for cls in range(len(class_labels)):
        print('{:<15} {:9} символов, {:8} слов'.format(class_labels[cls],
                                                         len(texts[cls]),
                                                         len(sequences[cls])))

        chars += len(texts[cls])
        words += len(sequences[cls])

    print('----')
    print('{:<15} {:9} символов, {:8} слов\n'.format('В сумме', chars, words))

# Вывод итогов по текстам
print_text_stats('обучающим', text_train, seq_train)
print_text_stats('тестовым', text_test, seq_test)
```

Статистика по обучающим текстам:

Стругацкие	2042470	символов,	313012	слов
Макс Фрай	3700011	символов,	568533	слов
Булгаков	1765649	символов,	261465	слов
Клиффорд Саймак	1609508	символов,	251502	слов

Пример. Классификация текстов

```
# Функция разбиения последовательности на отрезки скользящим окном
# На входе - последовательность индексов, размер окна, шаг окна
def split_sequence(sequence, win_size, hop):
    # Последовательность разбивается на части до последнего полного окна
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]

# Функция формирования выборок из последовательностей индексов
# формирует выборку отрезков и соответствующих им меток классов в виде one hot encoding
def vectorize_sequence(seq_list, win_size, hop):
    # В списке последовательности следуют в порядке их классов
    # Всего последовательностей в списке ровно столько, сколько классов
    class_count = len(seq_list)

    # Списки для исходных векторов и категориальных меток класса
    x, y = [], []

    # Для каждого класса:
    for cls in range(class_count):
        # Разбиение последовательности класса cls на отрезки
        vectors = split_sequence(seq_list[cls], win_size, hop)
        # Добавление отрезков в выборку
        x += vectors
        # Для всех отрезков класса cls добавление меток класса в виде ONE
        y += [utils.to_categorical(cls, class_count)] * len(vectors)

    # Возврат результатов как numpy-массивов
    return np.array(x), np.array(y)
```

Пример. Классификация текстов

```
# Формирование обучающей и тестовой выборки
# Формирование обучающей выборки
x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
# Формирование тестовой выборки
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

# Проверка формы сформированных данных
print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)

(17640, 1000) (17640, 6)
(6686, 1000) (6686, 6)
```

```
# Вывод отрезка индексов тренировочной выборки
print(x_train[0])
```

[	473	22	1	1496	47	2	12294	7581	5	414	1	4
	251	2	4	249	55	6	240	9134	2609	182	5765	1
	99	3506	18830	7	1	11	1	1	10	67	301	116
	34	4	1	18831	196	12	10855	1	18832	1	8682	1
	11508	1034	175	7267	13217	2	49	97	37	1	2	1
	6	55	17013	2	3362	137	45	1	13218	4861	6	32
	84	1	18833	5	4	106	1783	5766	2093	5766	17014	11
	7	1	12295	9662	1367	132	32	84	1	2	41	27
	4	3437	1	27	36	1367	132	3035	18834	2	2834	108
	521	4	5988	12	72	18835	12	72	1	46	12	72
	32	1	7	1560	774	1	1	1	7268	1	8	6482
	4591	18836	3665	83	4465	3	1	2342	1	2738	5561	2875
	4242	1	3	1	1	7	3363	1	17	368	31	86
	1	2876	386	2	1	17	7582	33	3824	2	4862	1
	100	68	3095	13	5197	55	4	1	2201	6	12	3
	279	15510	2	3745	1	137	12	952	652	1	5	51
	17015	15511	67	1	9	3364	569	317	1192	3507	2	166
	769	11	1609	5036	16	64	1409	67	770	1	35	36
	1	1441	1200	16	1	20	68	16	75	60	3095	135
	2202	4862	15512	1	51	3	1	1442	67	1	9135	18836
	17016	113	1	2782	2	13219	1741	241	1310	1966	241	3365
	4015	7	1	5767	39	17	144	371	211	7	4343	17017
	6255	241	1310	1966	9	36	6	12	56	1	2	1
	10	22	23	2343	1	1	154	1682	1	36	10856	86

Пример. Классификация текстов

```
# Формирование выборок текстов в виде разреженных матриц (bag of words)
# На входе .sequences_to_matrix() ожидает список, .tolist() выполняет преобразование типа
x_train_01 = tokenizer.sequences_to_matrix(x_train.tolist())
x_test_01 = tokenizer.sequences_to_matrix(x_test.tolist())
```

```
# Вывод формы обучающей выборки в виде разреженной матрицы Bag of Words
print(x_train_01.shape)
```

```
# Вывод фрагмента отрезка обучающего текста в виде Bag of Words
print(x_train_01[0][0:100])
```

```
(17640, 20000)
```

```
[0. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 0. 1. 1. 1. 0. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 0. 0. 0. 1. 1. 0. 1. 1. 1. 1. 0. 1. 1. 0. 0. 1. 1. 1.
 1. 0. 0. 1. 1. 0. 0. 1. 1. 1. 0. 1. 1. 1. 1. 0. 0. 0. 0. 0. 1. 0. 1.
 1. 1. 1. 1.]
```

Пример. Классификация текстов

BagOfWords

```
# Создание последовательной модели нейросети
model_text_bow_softmax = Sequential()
# Первый полносвязный слой
model_text_bow_softmax.add(Dense(200, input_dim=VOCAB_SIZE, activation="relu"))
# Слой регуляризации Dropout
model_text_bow_softmax.add(Dropout(0.25))
# Слой пакетной нормализации
model_text_bow_softmax.add(BatchNormalization())
# Выходной полносвязный слой
model_text_bow_softmax.add(Dense(CLASS_COUNT, activation='softmax'))

# Входные данные подаются в виде векторов bag of words
compile_train_eval_model(model_text_bow_softmax,
                        x_train_01, y_train,
                        x_test_01, y_test,
                        class_labels=CLASS_LIST,
                        title='BoW')
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 200)	4000200
dropout (Dropout)	(None, 200)	0
batch_normalization (Batch Normalization)	(None, 200)	800
dense_1 (Dense)	(None, 6)	1206

=====

Total params: 4,002,206

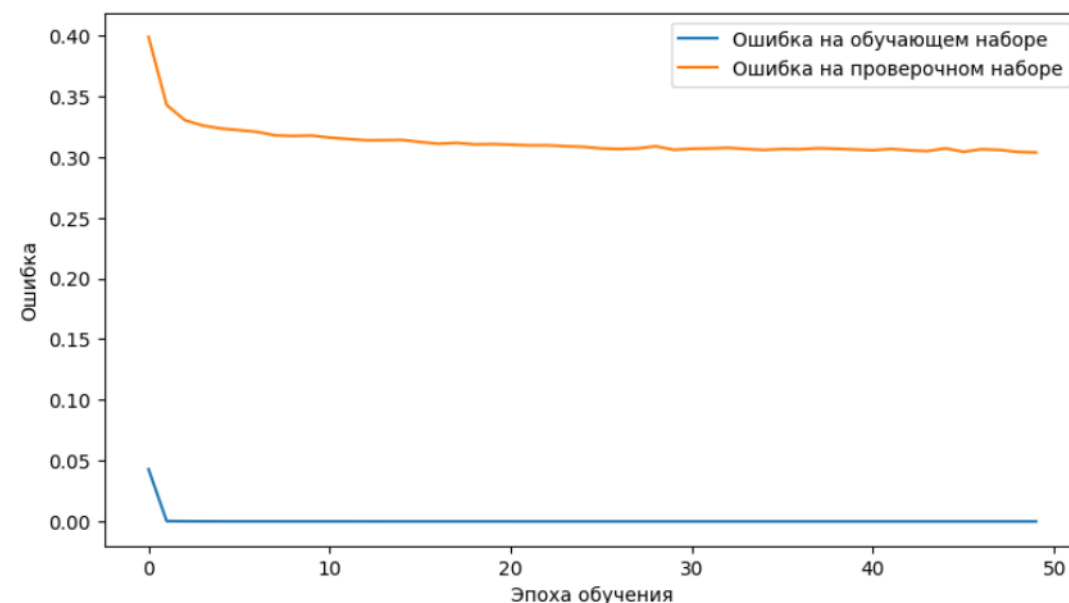
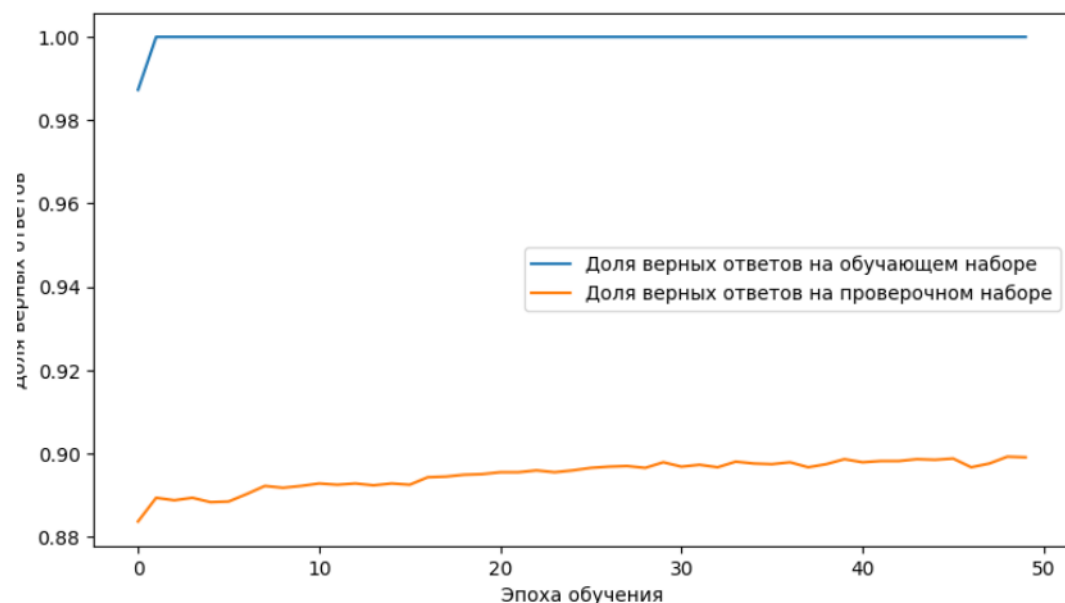
Trainable params: 4,001,806

Non-trainable params: 400

Пример. Классификация текстов

BagOfWords

График процесса обучения модели



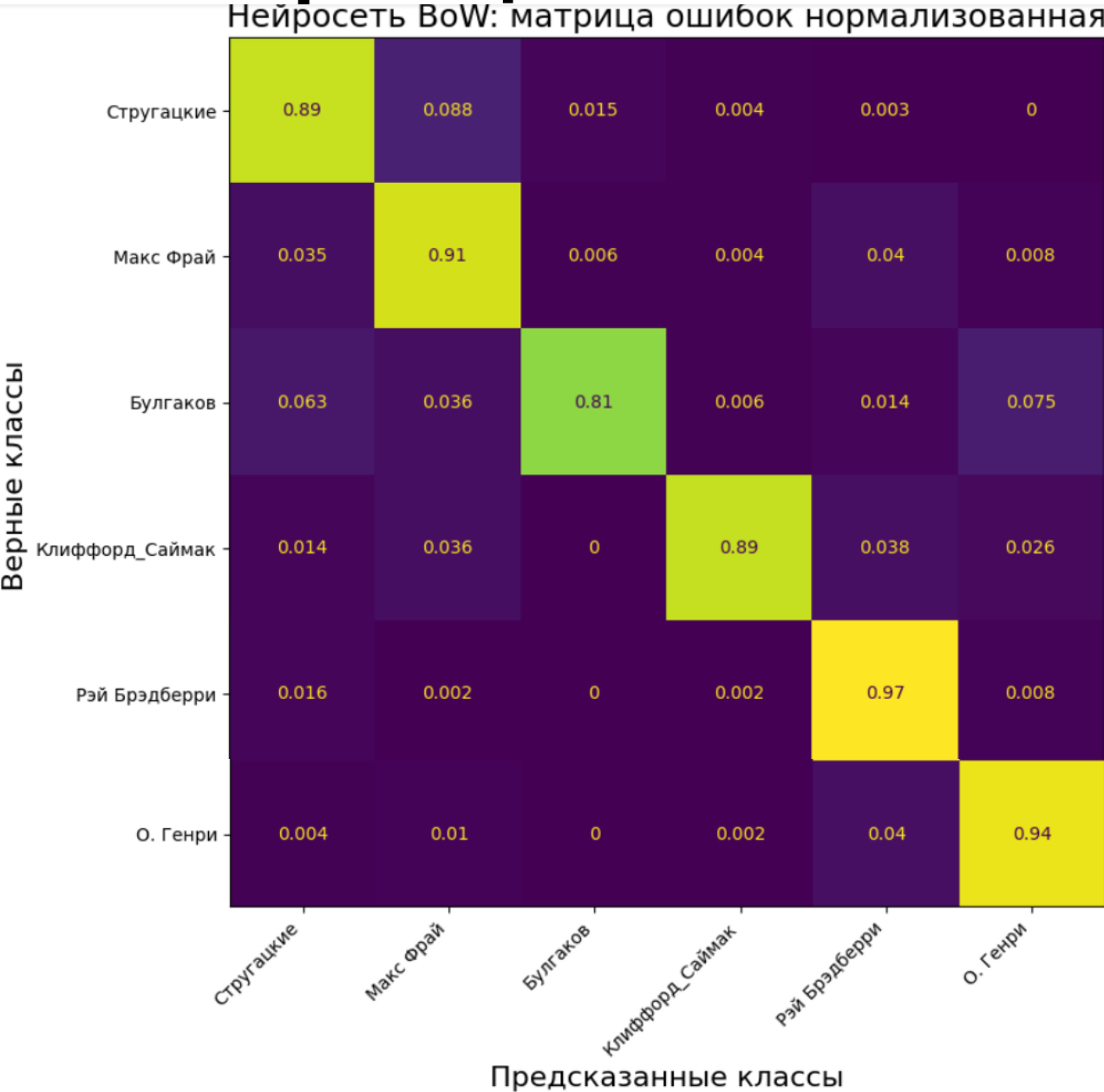
Нейросеть: BoW

Класс: Стругацкие	89% сеть отнесла к классу Стругацкие	- ВЕРНО :-)
Класс: Макс Фрай	91% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Булгаков	81% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Клиффорд Саймак	88% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: Рэй Брэдберри	97% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: О. Генри	94% сеть отнесла к классу О. Генри	- ВЕРНО :-)

Средняя точность распознавания: 90%

Пример. Классификация текстов

BagOfWords



Пример. Классификация текстов

Embedding

Embedding + Dense: размерность эмбедингов 20

```
[ ] # Архитектура со слоем Embedding и регуляризацией
    model_text_emb_20 = Sequential()
    model_text_emb_20.add(Embedding(input_dim = VOCAB_SIZE, output_dim = 20, input_length=WIN_SIZE))
    model_text_emb_20.add(SpatialDropout1D(0.2))
    model_text_emb_20.add(Flatten())
    model_text_emb_20.add(BatchNormalization())
    model_text_emb_20.add(Dense(200, activation="relu"))
    model_text_emb_20.add(Dropout(0.2))
    model_text_emb_20.add(BatchNormalization())
    model_text_emb_20.add(Dense(CLASS_COUNT, activation='softmax'))

    # Входные данные подаются в виде последовательностей индексов,
    # а не векторов bag of words
    compile_train_eval_model(model_text_emb_20,
                             x_train, y_train,
                             x_test, y_test,
                             class_labels=CLASS_LIST,
                             title='Embedding/20')
```

Пример. Классификация текстов

Embedding

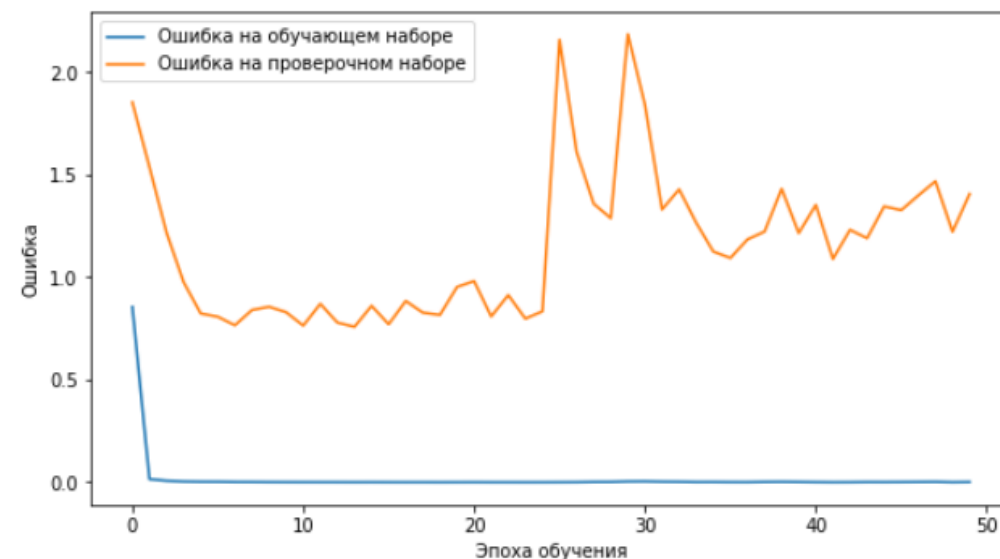
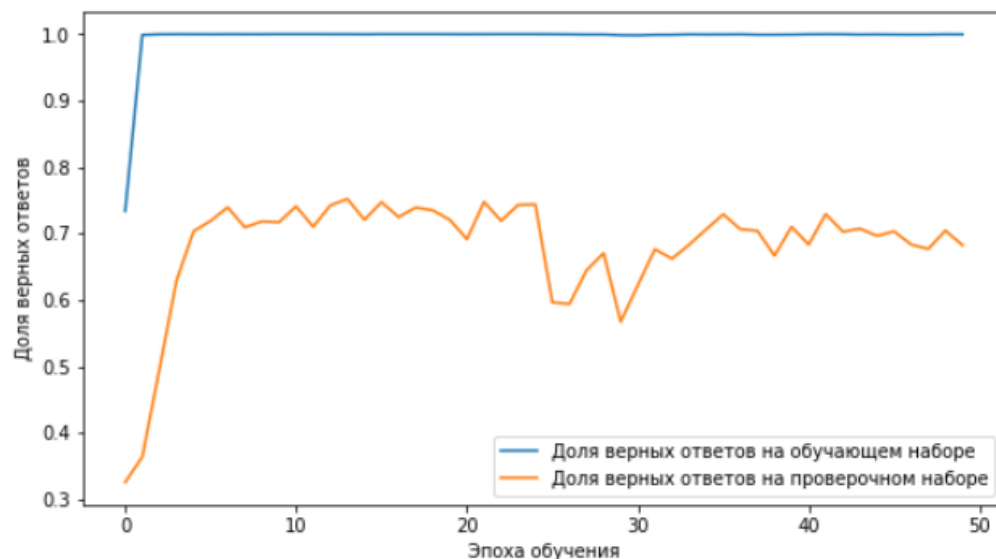
Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 1000, 20)	400000
spatial_dropout1d (SpatialD ropout1D)	(None, 1000, 20)	0
flatten (Flatten)	(None, 20000)	0
batch_normalization_1 (Batc hNormalization)	(None, 20000)	80000
dense_2 (Dense)	(None, 200)	4000200
dropout_1 (Dropout)	(None, 200)	0
batch_normalization_2 (Batc hNormalization)	(None, 200)	800
dense_3 (Dense)	(None, 6)	1206

Пример. Классификация текстов

Embedding

График процесса обучения модели



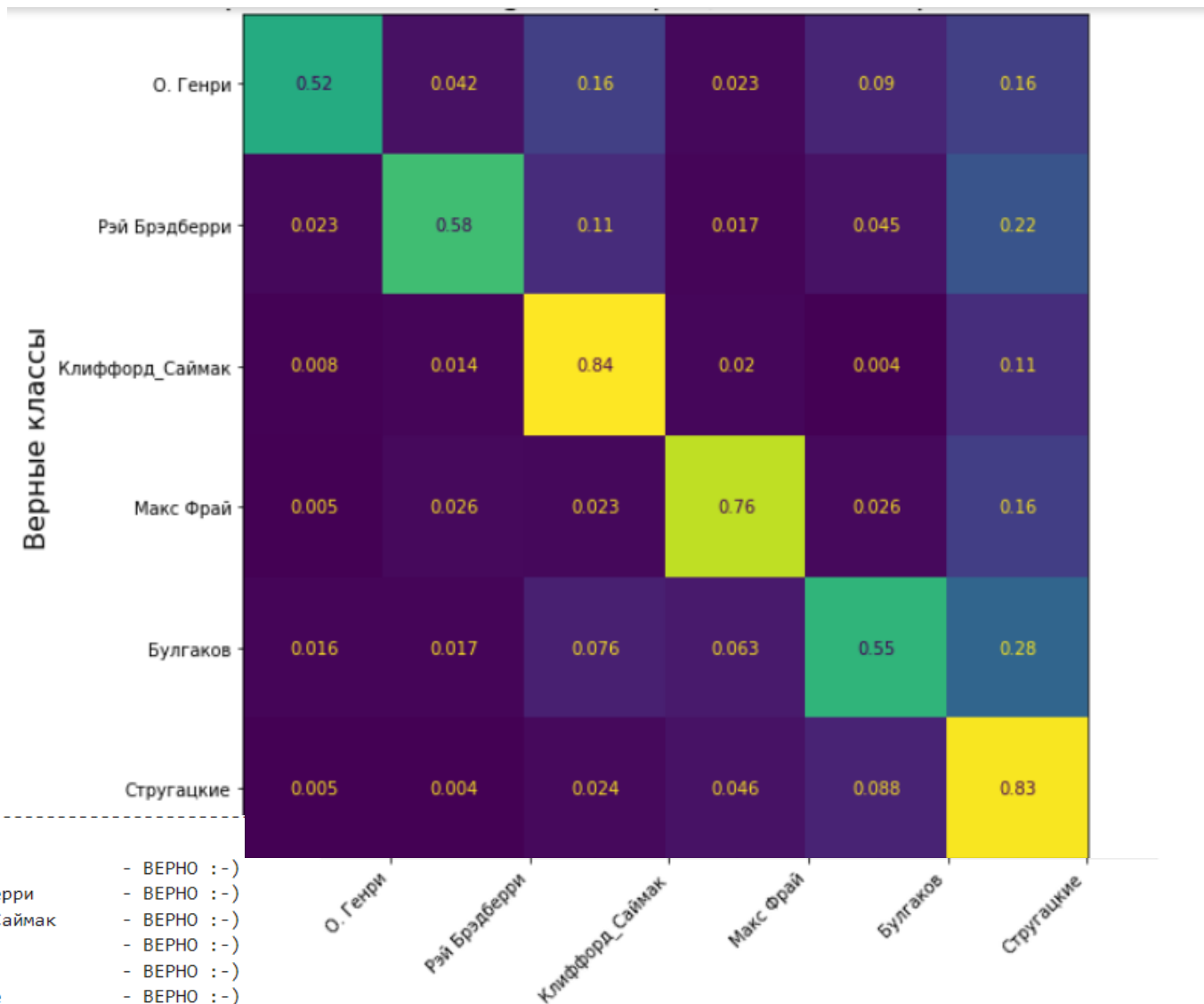
Нейросеть: Embedding/20

Класс: О. Генри	52% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Рэй Брэдберри	58% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	84% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: Макс Фрай	76% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Булгаков	55% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	83% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 68%

Пример. Классификация текстов

Embedding



Нейросеть: Embedding/20

Класс: О. Генри	52% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Рэй Брэдберри	58% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	84% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: Макс Фрай	76% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Булгаков	55% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	83% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 68%

KerasTuner

KerasTuner — это простая в использовании, масштабируемая структура оптимизации гиперпараметров,

Алгоритмы оптимизации гиперпараметров

RandomSearch - случайный поиск.

BayesianOptimization - Байесовская оптимизация

Hyperband - алгоритм оптимизации на основе многорукого бандита

Sequential Model-based Algorithm Configuration (SMAC),

Tree-structure Parzen Estimator (TPE),

Spearmint – деревья случайного поиска.

Встроенные алгоритмы

1. RandomSearch - случайный поиск.

2. BayesianOptimization - Байесовская оптимизация

3. Hyperband - алгоритм оптимизации на основе многорукого бандита

Официальный сайт: https://keras.io/keras_tuner/

Li, Lisha, and Kevin Jamieson Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization."Journal of Machine Learning Research 18 (2018): 1-52.

KerasTuner

```
pip install keras-tuner --upgrade
```

```
import keras_tuner
from tensorflow import keras
```

2. Функция создание модели Keras, hp - параметр для определения гиперпараметров во время создания модели.

```
def build_model(hp):
    model = keras.Sequential()
    model.add(keras.layers.Dense(
        hp.Choice('units', [8, 16, 32]),
        activation='relu'))
    model.add(keras.layers.Dense(1, activation='relu'))
    model.compile(loss='mse')
    return model
```

3. Инициализация тюнера (RandomSearch)

```
tuner = keras_tuner.RandomSearch(
    build_model,
    objective='val_loss',
    max_trials=5)
```

4. Запуск функции и поиск гиперпараметров

```
tuner.search(x_train, y_train, epochs=5, validation_data=(x_val, y_val))
best_model = tuner.get_best_models()[0]
```

KerasTuner

```
def build_model(hp):
    model = Sequential()
    activation_choice = hp.Choice('activation', values=['relu', 'sigmoid', 'tanh', 'elu', 'selu'])
    model.add(Dense(units=hp.Int('input',
                                min_value=512,      # мин кол-во нейронов - 512
                                max_value=1024,     # макс кол-во - 1024
                                step=128),
                    input_dim=784,
                    activation=activation_choice))
    model.add(Dense(10, activation='softmax'))
    model.compile(
        optimizer=hp.Choice('optimizer', values=['adam', 'rmsprop', 'SGD']),
        loss='categorical_crossentropy',
        metrics=['accuracy'])
    return model

tuner = RandomSearch(
    build_model,
    objective='val_accuracy',      # метрика оптимизации
    max_trials=40,                 # максимальное количество запусков обучения
    directory='test_directory'    # сохранение моделей
)

tuner.search_space_summary()
```

KerasTuner

```
def build_model(hp):
    model = Sequential()
    activation_choice = hp.Choice('activation', values=['relu', 'sigmoid', 'tanh', 'elu', 'selu'])
    model.add(Dense(units=hp.Int('input',
                                min_value=512,      # мин кол-во нейронов - 512
                                max_value=1024,     # макс кол-во - 1024
                                step=128),
                    input_dim=784,
                    activation=activation_choice))
    model.add(Dense(10, activation='softmax'))
    model.compile(
        optimizer=hp.Choice('optimizer', values=['adam', 'rmsprop', 'SGD']),
        loss='categorical_crossentropy',
        metrics=['accuracy'])
    return model

tuner = RandomSearch(
    build_model,
    objective='val_accuracy',      # метрика оптимизации
    max_trials=40,                 # максимальное количество запусков обучения
    directory='test_directory'    # сохранение моделей
)

tuner.search_space_summary()
```